08 - Clases abstractas, interfaces, enums y anidadas

Pregunta número 1

¿Cuál de las siguientes declaraciones de clase no es correcta?

Seleccione una:

- a. abstract class Person { }
- b. final class Person { }
- c. class Person { }
- d. abstract final class Person { }
- e. public abstract class Person { }

Respuesta: En este ejercicio se evalúa la compatibilidad de los modificadores de clase en Java. Dentro de los incisos hay uno que utiliza dos modificadores que son mutuamente excluyentes: abstract y final. Por un lado abstract indica que la clase es abstracta, lo que significa que está diseñada para ser heredada (extendida) por otras clases. No se puede instanciar directamente. Y por el otro lado, final indica que la clase es final, lo que significa que no puede ser heredada por ninguna otra clase.
Es una contradicción lógica: una clase no puede ser al mismo tiempo "obligatoria de heredar" (abstract) y "prohibida de heredar" (final). El compilador de Java lanzará un error de compilación si se intentan combinar estos dos modificadores.

Pregunta número 2

Which two statements are true?

Seleccione una o más de una:

- [x] a. A class that is abstract cannot be instantiated or turned into an object (instance).
- [ ] b. An interface cannot be extended by another interface.
- [x] c. If a class has abstract methods, then it must be specified as an abstract class.
- [ ] d. An abstract class can be extended by an interface.

Respuesta: Hay que analizar cada afirmación para determinar si es verdadera o falsa. Empecemos con el inciso a. A class that is abstract cannot be instantiated or turned into an object (instance). Por definición, una clase abstracta (abstract class) no se puede instanciar directamente usando la palabra clave new. Su propósito es servir como una plantilla base para que otras clases la extiendan (hereden) e

implementen sus métodos. Si intentamos hacer new ClaseAbstracta(), el compilador lanzará un error; por lo tanto, el inciso a es correcto. Vamos con el inciso b. An interface cannot be extended by another interface. Las interfaces sí pueden extenderse entre sí utilizando la palabra clave extends. De hecho, una interfaz puede extender múltiples interfaces a la vez (ej. interface C extends A, B {}). Lo que no puede hacer una interfaz es extender una clase; por lo tanto, el inciso b es falso.
Toca el inciso c. If a class has abstract methods, then it must be specified as an abstract class. Esta es una regla fundamental de Java. Si una clase contiene al menos un método abstracto (un método declarado sin cuerpo, usando la palabra clave abstract), la clase en sí misma debe ser declarada como abstract. De lo contrario, el código no compilará; por lo cual, este inciso es correcto. Finalmente, terminemos con el inciso d. An abstract class can be extended by an interface. La relación es incorrecta. Las interfaces no extienden clases. Lo que sí puede ocurrir es que una clase (incluida una clase abstracta) implemente una interfaz. La sintaxis correcta sería abstract class MiClase implements MiInterfaz {}. Una interfaz no puede "extender" una clase, ya sea abstracta o concreta.

Pregunta número 3

Which two statements are true?

Seleccione una o más de una:

- [ ] a. An interface CANNOT be extended by another interface
- [ ] b. An abstract class CANNOT be extended by an abstract class
- [x] c. An abstract class can be extended by a concrete class.
- [x] d. An abstract class can implement an interface
- [ ] e. An abstract class can be extended by an interface
- [ ] f. An interface can be extended by an abstract class

Respuesta: Hay que analizar cada afirmación sobre las reglas de herencia e interfaces en Java para saber cuál es correcta y cuál no. Comencemos con el inciso a. An interface CANNOT be extended by another interface. Las interfaces sí pueden extenderse entre sí usando extends. De hecho, una interfaz puede extender múltiples interfaces a la vez (ej. interface C extends A, B {}); por lo que esta respuesta es incorrecta. Seguimos con el inciso b. An abstract class CANNOT be extended by an abstract class. Las clases abstractas sí pueden heredar de otras clases abstractas usando extends; por lo cual, este inciso es incorrecto. Seguimos con el inciso c. An abstract class can be extended by a concrete class. Este es el propósito principal de una clase abstracta. Una clase concreta (no abstracta) puede

heredar de una clase abstracta utilizando la palabra clave extends. Si la clase abstracta tiene métodos abstractos, la clase concreta está obligada a implementarlos. Si no los implementa, la clase concreta también debe ser declarada como abstracta. Por lo tanto, el inciso c es correcto. El siguiente inciso es el d. An abstract class can implement an interface. Una clase abstracta puede implementar una interfaz utilizando la palabra clave implements. La ventaja es que la clase abstracta no está obligada a implementar todos los métodos de la interfaz; puede dejarlos como abstractos para que las subclases concretas los implementen más adelante. Por lo tanto, el inciso d es correcto. Vamos con el inciso e. An abstract class can be extended by an interface. Las interfaces no pueden extender clases. La relación correcta sería que una interfaz puede ser implementada por una clase (abstracta o no). El uso de "extended" aquí es incorrecto, por lo que el inciso e es incorrecto. Finalmente, terminemos con el inciso f. An interface can be extended by an abstract class. Al igual que vimos en la opción e, el término correcto es que una clase abstracta implementa (implements) una interfaz, no la extiende (extends). Una interfaz solo puede ser extendida por otra interfaz, por lo que el inciso f es incorrecto.

Pregunta número 4

Whats is true about the class Wow?

```
public abstract class Wow {
    private int wow;
    public Wow(int wow) { this.wow = wow; }
    public void wow() { }
    private void wowza() { }
}
```

Seleccione una:

- a. It compiles without error.
- b. It does not compile because an abstract class cannot have private methods
- c. It does not compile because an abstract class cannot have instance variables.
- d. It does not compile because an abstract class must have at least one abstract method.
- e. It does not compile because an abstract class must have a constructor with no arguments.

Respuesta: Debemos recordar que en Java, las clases abstractas son clases que no se pueden instanciar directamente, pero que pueden contener una mezcla de métodos abstractos y concretos, así como variables de instancia y constructores.

Hay que analizar el código proporcionado para verificar por qué es válido. Empecemos con public abstract class Wow: Declara correctamente una clase abstracta. Después tenemos private int wow: Las clases abstractas sí pueden tener variables de instancia (incluidas las privadas). Seguimos con public Wow(int wow) { ... }: Las clases abstractas sí pueden tener constructores con argumentos. Aunque no se puede hacer new Wow(5), las subclases que hereden de Wow podrán llamar a este constructor usando super(wow). Continuamos con public void wow() { }: Pueden tener métodos concretos (con implementación). Finalmente, llegamos a la línea private void wowza() { }: Las clases abstractas sí pueden tener métodos privados. Estos métodos no pueden ser sobrescritos por las subclases, pero pueden ser utilizados internamente por la propia clase; por lo tanto, el código compila sin errores.

Pregunta número 5

Which statement is true about the following code?

```
abstract class Shape {
    public abstract void draw();
    public void printShape() {
        System.out.println("This is a shape");
    }
}
class Circle extends Shape {
    @Override
    public void draw() {
        System.out.println("Drawing a circle");
    }
}
public class Test {
    public static void main(String[] args) {
        Circle circle = new Circle();
        circle.draw();
        circle.printShape();
    }
}
```

Seleccione una:

- a. The program will print "Drawing a circle" followed by "This is a shape".
- b. Compilation error because the Shape class is abstract.
- c. The program will print "This is a shape" followed by "Drawing a circle".
- d. Compilation error because the Circle class does not implement the draw method.

Respuesta: En este ejercicio se nos demuestra el uso correcto de una clase abstracta, la herencia y la sobrescritura de métodos. Vayamos paso a paso por la

ejecución para entender lo que está sucediendo: Empezamos con la definición de la clase abstracta Shape: Se declara un método abstracto public abstract void draw(). Esto significa que cualquier clase que herede de Shape está obligada a proporcionar una implementación para draw(). También tiene un método concreto public void printShape(), que imprime "This is a shape". Seguimos con la definición de la clase Circle (class Circle extends Shape): Hereda de la clase abstracta. @Override public void draw(): Implementa el método abstracto. Su cuerpo imprime "Drawing a circle". Vayamos a la ejecución en el método main (Circle circle = new Circle()). Se crea una instancia de la clase concreta Circle. Esto es válido porque Circle implementa todos los métodos abstractos de Shape. Después, circle.draw() llama al método draw() de Circle. Se imprime "Drawing a circle". Finalizamos con circle.printShape(): Llama al método heredado de Shape. Se imprime "This is a shape".

Pregunta número 6

What will be the output of the following code?

```
abstract class Vehicle {
    public abstract void start();

    public void stop() {
        System.out.println("Vehicle stopped");
    }
}

class Car extends Vehicle {
    @Override
    public void start() {
        System.out.println("Car started");
    }
}

class Bike extends Vehicle {
    @Override
    public void start() {
        System.out.println("Bike started");
    }
}

public class Test {
    public static void main(String[] args) {
        Vehicle car = new Car();
        Vehicle bike = new Bike();
        car.start();
        car.stop();
        bike.start();
        bike.stop();
    }
}
```

Seleccione una:

- a. Car started Vehicle stopped Bike started Vehicle stopped
- b. Vehicle stopped Car started Vehicle stopped Bike started
- c. Bike started Vehicle stopped Car started Vehicle stopped
- d. Car started Vehicle stopped Vehicle stopped Bike started

Respuesta: En este ejercicio se evalúa el comportamiento del polimorfismo y la herencia de métodos concretos desde una clase abstracta. Vayamos desglosando la ejecución paso a paso en el método main para entender qué está pasando. Empecemos con car.start(): Gracias al despacho dinámico de métodos (polimorfismo), ejecuta la implementación de la subclase Car, imprimiendo "Car started". Seguimos con car.stop(): Al no estar sobrescrito en Car, ejecuta el método stop() heredado de la superclase Vehicle, imprimiendo "Vehicle stopped". Continuamos con bike.start(): Ejecuta la implementación de la subclase Bike, imprimiendo "Bike started". Finalmente, tenemos bike.stop(): Ejecuta el método stop() heredado de Vehicle, imprimiendo "Vehicle stopped".

Pregunta número 7

What will be the output of the following code snippet?

```
public class EnumTest {
    enum Day {
        MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
    }
    public static void main(String[] args) {
        for (Day day : Day.values()) {
            System.out.print(day + " ");
        }
    }
}
```

Seleccione una:

- a. Compilation error.
- b. MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY
- c. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY
- d. MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY

Respuesta: En este ejercicio se utiliza un bucle for-each para recorrer todos los valores de un tipo enum llamado Day. Vayamos paso a paso para entender qué está

pasando. Empecemos con la obtención de constantes: El método sintético Day.values() devuelve un arreglo con todas las constantes del enum en el orden exacto en que fueron declaradas (MONDAY a SUNDAY). Seguimos con el formato de impresión: La instrucción System.out.print(day + " ") convierte cada valor a su representación en texto (toString()) y le concatena un espacio en blanco en lugar de una coma (descartando la opción b). Finalmente, al utilizar print en lugar de println, todos los elementos se imprimen sucesivamente en la misma línea, abarcando los 7 elementos del enum.

Pregunta número 8

Which of the following code snippets will result in a compilation error when implementing the Vehicle interface?

```
interface Vehicle {
    void start();
    void stop();
}
```

Seleccione una:

a.

```
Opción a:
public class Scooter implements Vehicle {
    public void start() {
        System.out.println("Scooter starts");
    }
    public void stop() {
        System.out.println("Scooter stops");
    }
}
```

b.

```
Opción b:
public class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike starts");
    }
    void stop() {
        System.out.println("Bike stops");
    }
}
```

c.

```
Opción c:
public class Truck implements Vehicle {
    public void start() {
        System.out.println("Truck starts");
    }
    public void stop() {
        System.out.println("Truck stops");
    }
    public void load() {
        System.out.println("Truck loads");
    }
}
```

d.

```
Opción d:
public class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts");
    }
    public void stop() {
        System.out.println("Car stops");
    }
}
```

Respuesta: Primero tenemos que recordar que en Java, todos los métodos declarados en una interfaz son implícitamente public y abstract. Cuando una clase

implementa una interfaz; debe proporcionar una implementación para todos sus métodos, y el método que implementa debe ser declarado explícitamente como public. No se puede reducir la visibilidad de un método al sobrescribirlo (esto se conoce como "no se pueden asignar privilegios de acceso más débiles"). Conociendo la regla, ahora sí podemos analizar las opciones para ver cuál es la incorrecta: Empecemos con el inciso a. Ambos métodos (start y stop) están declarados como public, cumpliendo con el contrato de la interfaz, por lo cual este inciso es correcto. Seguimos con el inciso b. El método stop() está declarado sin modificador de acceso, lo que en Java significa que tiene acceso por defecto (o package-private). Esto es un nivel de acceso más restrictivo que public. Al intentar implementar el método stop() de la interfaz (que es público) con un método que no es público, el compilador lanzará un error indicando que se están intentando asignar privilegios de acceso más débiles, por lo cual este es el inciso incorrecto, pero continuemos con los otros incisos para ver por qué los otros 2 incisos son correctos. Ahora tenemos el inciso c. Implementa correctamente start y stop como public. Además, añade un método extra load(), lo cual es perfectamente válido y no causa errores. Finalmente, tenemos el inciso d. Al igual que la opción a, implementa ambos métodos requeridos como public, por lo que este inciso es correcto.

Pregunta número 9

Which code fragment is illegal?

Seleccione una:

- a. `class Base1 { abstract class Abs1 { } }`
- b. `abstract class Abs2 { void doit() { } }`
- c. `class Base2 { abstract class Abs3 extends Base2 { } }`
- d. `class Base3 { abstract int var1 = 89; }`

Respuesta: Conociendo el uso correcto del modificador abstract podemos saber cuál es la respuesta correcta rápidamente. Recordemos que en Java, el modificador abstract solo se puede aplicar a clases, interfaces y métodos. Nunca se puede aplicar a variables (sean de instancia, estáticas o locales). Intentar declarar una variable como abstract produce un error de compilación inmediato. Con esto en mente, podemos afirmar que el inciso que tenemos que elegir es el d, pero hay que ver por qué las otras opciones sí son válidas. Empecemos con el inciso a: Es completamente válido definir una clase anidada/interna abstracta dentro de una clase concreta. Continuemos con el inciso b: Una clase abstracta puede contener métodos concretos (con cuerpo) sin necesidad de tener métodos abstractos. Finalmente, llegamos al inciso c: Una clase interna abstracta puede extender a su propia clase contenedora exterior.

Pregunta número 10

Given: Which method can be declared within the C class to make it compile?

```
interface I {
    void method();
}
public class C implements I {
// method declarations
}
```

Seleccione una:

- a. public abstract void method();
- b. protected abstract void method() { }
- c. private void method() {}
- d. void method() {}
- e. None of the options is valid.

Respuesta: Para que la clase C compile correctamente, debe implementar el método method() de la interfaz I. Esto nos conlleva dos reglas fundamentales en Java. La primera es la visibilidad: Los métodos en una interfaz son implícitamente public. Cuando una clase implementa un método de una interfaz, debe declararlo explícitamente como public. No se puede reducir su visibilidad. La segunda es la concreción: Dado que C es una clase concreta (public class C), no puede contener métodos abstractos. Debe proporcionar un cuerpo (implementación) para el método. Tenemos que analizar por qué cada opción falla: Empecemos con el inciso a. public abstract void method(). Aunque es public, es abstract. Una clase concreta no puede tener métodos abstractos. Esto daría un error de compilación indicando que la clase debe ser declarada como abstracta o implementar el método. Seguimos con el inciso b. protected abstract void method() { }. Esta opción tiene dos errores. Primero, usa protected en lugar de public, lo cual reduce la visibilidad y causa un error. Segundo, declara el método como abstract pero le proporciona un cuerpo ({ }), lo cual es contradictorio y sintácticamente inválido. Seguimos con el inciso c. private void method() { }. Primero que nada, usa private. Al intentar implementar un método public de la interfaz con uno private, el compilador lanza un error por intentar asignar privilegios de acceso más débiles. Finalmente, tenemos el inciso d. void method() { }. Este no tiene modificador de acceso, lo que significa que tiene acceso por defecto (package-private). Al igual que en la opción c, esto es menos visible que public y

provoca un error de compilación. Con lo anterior dicho, podemos decir que ningún inciso da una opción correcta.

Pregunta número 11

What is the result?

```
interface Rideable {
    String getGait();
}

public class Camel implements Rideable {
    int weight = 2;
    String getGait() {
        return " mph, lope";
    }
    void go(int speed) {
        ++speed;
        Weight++;
        int walkrate = speed * weight;
        System.out.print(walkrate + getGait());
    }
    public static void main(String[] args) {
        new Camel().go(8);
    }
}
```

Seleccione una:

- a. 16 mph, lope
- b. 24 mph, lope.
- c. 27 mph, lope.
- d. Compilation fails

Respuesta: En este ejercicio tenemos dos errores en la clase Camel que evitan que el código compile; analicemos con cuidado para entender por qué está pasando esto. Primero tenemos el error de visibilidad en getGait(): La interfaz Rideable declara el método String getGait(). En Java, todos los métodos de una interfaz son

implícitamente public. Cuando una clase implementa un método de una interfaz, debe declararlo explícitamente como public. En el código, el método se declara como String getGait() (sin modificador de acceso, lo que significa acceso por defecto o package-private). Esto es un nivel de acceso más restrictivo que public, por lo que el compilador lanzará un error indicando que se están intentando asignar privilegios de acceso más débiles. La forma correcta sería: public String getGait() { ... }. El otro error es de mayúsculas en Weight++. La variable se declara como int weight = 2; (con 'w' minúscula). Dentro del método go, se intenta incrementar usando Weight++; (con 'W' mayúscula). Recordemos que Java es un lenguaje case-sensitive (distingue entre mayúsculas y minúsculas). La variable Weight no existe en el ámbito del método, lo que provocará un error de compilación del tipo "cannot find symbol" (no se encuentra el símbolo).

Pregunta número 12

Dado el siguiente código. ¿Cuál es el resultado?

```
abstract class A1{
    public abstract void m1();
    public void m2() {System.out.println("Verde");}
}

abstract class A2 extends A1 {
    public abstract void m3();
    public void m1() {System.out.println("Cafe");}
    public void m2() {System.out.println("Azul");}
}

public class A3 extends A2 {
    public void m1() {System.out.println("Amarillo");}
    public void m2() {System.out.println("Rosa");}
    public void m3() {System.out.println("Rojo");}
    public static void main(String[] args) {
        A2 tp = new A3();
        tp.m1();
        tp.m2();
        tp.m3();
    }
}
```

Seleccione una:

- a. Cafe Azul Rojo
- b. Amarillo Rosa Rojo
- c. Verde Azul Rojo
- d. Runtime exception

Respuesta: En este ejercicio se evalúa el polimorfismo y el mecanismo de invocación virtual de métodos (virtual method invocation) a través de una jerarquía de clases de múltiples niveles. Primero veamos la diferencia entre tipo de referencia vs. tipo de objeto real: En la línea A2 tp = new A3(), el tipo de referencia en tiempo de compilación es A2, pero la instancia real creada en la memoria Heap en tiempo de ejecución es de la subclase A3. Dado que los métodos m1(), m2() y m3() están declarados en la clase de referencia A2 (de forma abstracta o concreta), la llamada tp.m1(), tp.m2() y tp.m3() compila sin errores. Es importante recordar que en Java, la llamada a métodos de instancia no estáticos se resuelve dinámicamente según la clase del objeto real (A3) y no por el tipo de variable que lo almacena (A2): tp.m1() invoca la implementación en A3 → imprime Amarillo. tp.m2() invoca la implementación en A3 → imprime Rosa. tp.m3() invoca la implementación en A3 → imprime Rojo.

Pregunta número 13

What is the result?

```
class Ot {
    enum Colors {
        RED, GREEN, BLUE, YELLOW, BLACK
    };
}
public class Test {
    public static void main(String[] String) {
        for (Ot.Colors c : Ot.Colors.values()) {
            if (Ot.Colors.RED.equals(c)) {
                System.out.print("red ");
            }
            if (c == Ot.Colors.GREEN) {
                System.out.print("green ");
            }
            if (c.equals("BLUE")) {
                System.out.print("blue ");
            }
        }
    }
}
```

Seleccione una:

- a. red blue
- b. green blue
- c. red green blue
- d. red green
- e. An exception is thrown at runtime.
- f. Compilation fails.

Respuesta: En este ejercicio se demuestra cómo funcionan los tipos enum en Java, específicamente la comparación de sus constantes. Vayamos paso a paso por la ejecución del bucle for para entender qué está pasando. Empecemos con la iteración del enum: El método Ot.Colors.values() devuelve un arreglo con todas las constantes del enum en el orden en que fueron declaradas: RED, GREEN, BLUE, YELLOW, BLACK. El bucle for itera sobre cada una de estas constantes. Seguimos con la evaluación de las condiciones para cada color:
Primero tenemos RED: Ot.Colors.RED.equals(c) es verdadero (es la misma constante). Se imprime "red ". c == Ot.Colors.GREEN es falso. c.equals("BLUE") es falso (un enum no es igual a un String).
Vamos con GREEN: Ot.Colors.RED.equals(c) es falso. c == Ot.Colors.GREEN es verdadero. Se imprime "green ". c.equals("BLUE") es falso.
Seguimos con BLUE: Ot.Colors.RED.equals(c) es falso. c == Ot.Colors.GREEN es falso. c.equals("BLUE"): Aquí está la clave. La variable c es de tipo Ot.Colors (un objeto enum). El método .equals() se hereda de la clase Object y compara si el objeto pasado es el mismo. Un objeto Ot.Colors.BLUE nunca será igual a un objeto String "BLUE". Por lo tanto, esta condición es falsa. No se imprime nada.
Finalmente, llegamos a YELLOW y BLACK: Ninguna de las condiciones se cumple para estos colores, por lo que no se imprime nada. La salida final que tenemos es que el programa solo imprime "red" y "green", por lo que el resultado será red green.

Pregunta número 14

¿Cuáles son los modificadores que puede tener una clase anidada en Java?

Seleccione una:

- a. public
- b. final
- c. protected
- d. abstract
- e. static
- f. private
- g. strictfp
- h. Todas las opciones son correctas

Respuesta: Primero debemos recordar que en Java, una clase anidada (nested class) es aquella que se declara dentro de otra clase. A diferencia de las clases de nivel superior (top-level classes), las clases anidadas tienen permitido usar

prácticamente cualquier modificador de acceso y de no acceso. Analicemos por qué cada uno de los modificadores listados es válido para una clase anidada: Tenemos primero los modificadores de acceso:
public (a): La clase anidada es accesible desde cualquier lugar.
protected (c): La clase anidada es accesible desde el mismo paquete o desde subclases de la clase externa.
private (f): La clase anidada solo es accesible dentro del ámbito de la clase externa que la contiene. Esto es muy útil para ocultar implementaciones. Es importante saber que, si no se pone ningún modificador, tiene acceso por defecto o package-private.
Ahora vamos con los modificadores de no acceso:
static (e): Permite declarar una clase anidada estática. Esto significa que no necesita una instancia de la clase externa para ser instanciada.
final (b): Indica que la clase anidada no puede ser heredada (no puede tener subclases).
abstract (d): Indica que la clase anidada es abstracta y no puede ser instanciada directamente.
strictfp (g): Asegura que los cálculos de punto flotante dentro de esa clase sean portables entre plataformas.
La clave de la diferencia es que, mientras que una clase de nivel superior (como public class Test en un archivo .java) solo puede ser public o tener acceso por defecto, una clase anidada puede combinar modificadores de acceso (public, protected, private) con modificadores de no acceso (static, final, abstract, strictfp).